



Electron Control System

Electron Architecture

Scenario - Lite S/X

Scenario - Lite Ka

Spaceport

Electron and spaceport extensions

Electron - Scheduler Camriere and ...

Electron - Scheduler brokkr extensi...

Electron - ingest api extension

Electron - NAOS extension

Electron - Receptor gathering exte...

Spaceport - Monitor API with Cosm...

Spaceport - Lite Monitor and Contr...

Spaceport - SpaceNintendo GUI

MEOS Control Systems

Architecture

Falcon

Hypermatter

Girodyne

Observability

Grafana

OpenTelemetry

Deployment

GN Products Deployment Services

Deployment with AWX

Deployment to Openstack

Quick Guide - Inventory and Deplo...

How to make api key in KOGNI for Ele...

Vault

Spaceport Extension - Cosmic_monitor_api

This extension consists of the `backend_scraper` (s) and a `monitor_api_cosmic_integration` enabling monitor api at the antenna level through Cosmic. This extension can be used on systems with backend supported by the `backend_scraper`. As of April 2025, this is

Meos Control V4 and V5

Kratos qRadio, qRx, qRadio and qFEP

Mini Circuits zt272

Safran MCS

The specific containers in this extension its worth noting are

[backend_scraper \(minatore\)](#) - Scrapes one designated backend. Can be deployed in parallel to scrape multiple backends. It publishes metrics to Cosmic.

[monitor_api_cosmic_integration](#) - Monitor Api integration using metrics in Cosmic

The deployment of this extension is part of the path to deliver monitor and control api via cosmic.

The image below shows a general instance of Electron and how it together with Spaceport and this extension delivers monitor api v1 via Cosmic (Monitor api v2 is not detailed here, other than that it is provide by Cosmic.) As of 6th of June 2025, the lite legacy controller provides the contact id from kogs to the scraper, and sets up access proxy ahead of aos. The customer access monitor api through the site proxy configured by Marius Flage. This extension deploys the scraper and the monitor api v1 emulator to spaceport. The emulators purpose is to authenticate against Cosmic, fetch the metrics related to the contact, and provide it to the costumer such that the customer experience is the same as Monitor Api V1 detailed in [lite_monitor_control_api extension](#).

Spaceport Extension - Co...

backend_scraper

Short introduction to Min...

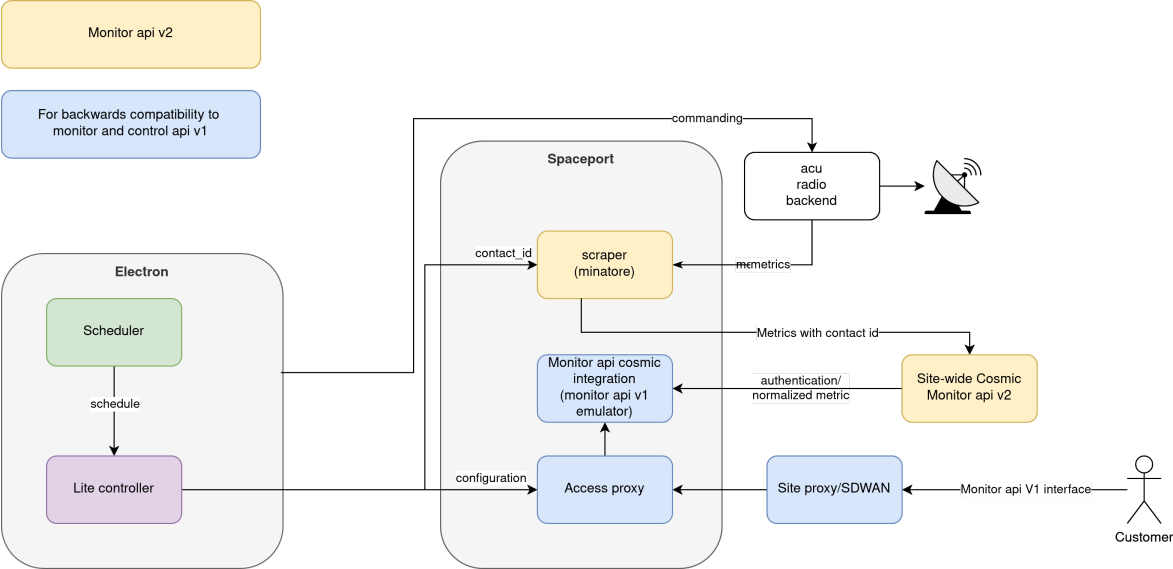
Deployment

How to find scraper_mo...

monitor_api_cosmic_integ...

Deployment

How to test



The end goal is to also provide control api with this extension. It is however not ready (6th june 2025).

backend_scraper

The `backend_scraper` is a software module called *Minatore*. Its main purpose is to scrape the backend for metrics continuously and publish the metrics to your desired dispatcher. It is compatible with Cosmic.

The source code and documentation can be found [here](#).

Short introduction to Minatore

Minatore aims to replace all scrapers and merge them into one easy maintainable scraper.

- Minatore can scrape i.e. Kratos or MEOS and publish it to Cosmic
- Environment variables to configure Minatore is set during deployment.
- Multiple instances can run to scrape different hardware, customizable with environment variables in inventory during deployment.
- Minatore aims to support multiple hardware and middlewares for metrics. See the source code documentation for compatible antenna equipment for scraping.

Deployment

Minatore is deployed with GSC Deployment services through the extension `cosmic_monitor_api` . One scraper instance can scrape one backend/mw. For MEOS Control MW, you need one scraper for the MEOS Control. For

Electron Control System

Electron Architecture

Scenario - Lite S/X

Scenario - Lite Ka

Spaceport

Electron and spaceport extensions

Electron - Scheduler Camriere and ...

Electron - Scheduler brokkr extensi...

Electron - ingest api extension

Electron - NAOS extension

Electron - Receptor gathering exte...

Spaceport - Monitor API with Cosm...

Spaceport - Lite Monitor and Contr...

Spaceport - SpaceNintendo GUI

MEOS Control Systems

Architecture

Falcon

Hypermatter

Girodyne

Observability

Grafana

OpenTelemetry

Deployment

GN Products Deployment Services

Deployment with AWX

Deployment to Openstack

Quick Guide - Inventory and Deplo...

How to make api key in KOGNI for Ele...

Vault

Michael.Johansen@ksat.no

Kratos equipment you need i.e one scraper for the qDRA, one for the qRX etc.

In the inventory, add the following variables.

Note! the environment variables can change from one version to another. Keep yourself up to date by inspecting the source code documentation, and the deployment template.

The deployer are responsible for providing the metrics to be scraped. In the example below, we show the two different ways to do this.

Required variables

VARIABLE	TYPE	DESCRIPTION
<code>system_id</code>	string	uuid from kogni
<code>scrapers</code>	list	list of scrapers.
The following sub - variables are needed under <code>scrapers</code>		
<code>scraper_mode</code>	string	Allowed values only. See docs
<code>scraper_device</code>	string	Unique name of scraper
<code>scraper_device_type</code>	string	See allowed values under Body -> device -> type
<code>scraper_device_version</code>	string	Version of scraped device (i.e the meos, qdra, qrx etc)
<code>scraper_scraped_host</code>	string	ip address of scraped device (i.e the meos, qdra, qrx etc)
<code>scraper_cosmic_host</code>	string	ip address of cosmic
<code>scraper_modules</code>	string	string of list of modules to scrape. Can also add "include vars option" see example below
<code>scraper_scraped_host_port</code>	string	defaults to port 10000 for meos, 8080 for kratos. define to overwrite

```
# example 1 meos antenna
ANT1:
  #spaceport vars
  extensions: backend_scraper
  system_id: 00000000-0000-0000-0000-000000000000 # uuid form kogni
  scrapers:
    - name: mcontrol_scraper
      scraper_mode: MEOS
```

Electron Control System

Electron Architecture

Scenario - Lite S/X

Scenario - Lite Ka

Spaceport

Electron and spaceport extensions

Electron - Scheduler Camriere and ...

Electron - Scheduler brokkr extensi...

Electron - ingest api extension

Electron - NAOS extension

Electron - Receptor gathering exte...

Spaceport - Monitor API with Cosm...

Spaceport - Lite Monitor and Contr...

Spaceport - SpaceNintendo GUI

MEOS Control Systems

Architecture

Falcon

Hypermatter

Girodyne

Observability

Grafana

OpenTelemetry

Deployment

GN Products Deployment Services

Deployment with AWX

Deployment to Openstack

Quick Guide - Inventory and Deplo...

How to make api key in KOGNI for Ele...

Vault

Michael.Johansen@ksat.no

```
scraper_device: ANT1-MCONTROL #unique name
scraper_device_type: spacetec-meos # see other allowed values in minatore docs
scraper_device_version: # backend verison. Here: the meos control version
scraper_scraped_host: # ip of backend device. Here: the meos control version
scraper_cosmic_host: # ip of cosmic on the site.
# the modules below need to be correct with the modules available on the meos. See in "
scraper_modules: >-
  MEOS.CORTEX_CART1.CRT1, MEOS.CORTEX_CART1.DCU1, MEOS.CORTEX_CART1.EFC1, MEOS.CORTEX_CART1
  MEOS.CORTEX_CART1.IFM1,
  MEOS.CORTEX_CART1.IFR1, MEOS.CORTEX_CART1.IFR2, MEOS.CORTEX_CART1.IFR3, MEOS.CORTEX_CART1
  MEOS.CORTEX_CART1.IMG1, MEOS.CORTEX_CART1.RAU1, MEOS.CORTEX_CART1.RAUX1, MEOS.CORTEX_CART1
  MEOS.CORTEX_CART1.SLE_GW_MAIN1, MEOS.CORTEX_CART1.SLE_GW_SIMON1, MEOS.CORTEX_CART1.SLE_G
  MEOS.CORTEX_CART1.TCU1, MEOS.CORTEX_CART1.TMS1,
  MEOS.CORTEX_CART1.TMU1, MEOS.CORTEX_CART1.TMU2, MEOS.CORTEX_CART1.TMU3,
```

secrets in vault

KEY	DESCRIPTION
backend_scraper_api_key	made in kogni. Must have Metrics - write role

How to find scraper_modules

We have made the bare minimum list of modules to provide the customer with the monitoring api for the most common meos control versions. If your system have another version, you might need to find the correct module names.

MEOS CONTROL

In meos control, click view -> history. Under subsystems, you have each module. You need to provide the path to each module under each subsystem. i.e, by providing scraper_modules: "MCONTROL.SAFRAN_PRO.ACU" will scrape all the metrics in this module.

Subsystems

MCONTROL

SAFRAN_PRO

SAFRAN_PRO

ACU

s_tx_polar_status

pass_plan_alarm

Kratos

Electron Control System

Electron Architecture

Scenario - Lite S/X

Scenario - Lite Ka

Spaceport

Electron and spaceport extensions

Electron - Scheduler Camriere and ...

Electron - Scheduler brokkr extensi...

Electron - ingest api extension

Electron - NAOS extension

Electron - Receptor gathering exte...

Spaceport - Monitor API with Cosm...

Spaceport - Lite Monitor and Contr...

Spaceport - SpaceNintendo GUI

MEOS Control Systems

Architecture

Falcon

Hypermatter

Girodyne

Observability

Grafana

OpenTelemetry

Deployment

GN Products Deployment Services

Deployment with AWX

Deployment to Openstack

Quick Guide - Inventory and Deplo...

How to make api key in KOGNI for Ele...

Vault

Michael.Johansen@ksat.no

Open the following in a browser `http://<ip-adress>:8080/rest` . Copy paste the contents in the resource list:

```
{ "_resource": [module1, module2] }
```

monitor_api_cosmic_integration

The `monitor_api_cosmic_integration` container is a software component to enable the Cosmic Monitor Api. The component is accessed through access proxy, giving the same integration for customers even when the data is stored in Cosmic.

The service emulates the old Maestro Monitoring-API where it replecates the endpoints that is available for the customers receive, telemetry, and spec. and connects them to the upstream Cosmic API service that has MEOS metrics.

Deployment

Monitor-api-integration is deployed with GSC Deployment services through the extension

```
cosmic_monitor_api
```

In the inventory, add the following variables. Note! the env vars can change with time. Keep yourself up to date by inspecting the source code documentation, and the deployment template.

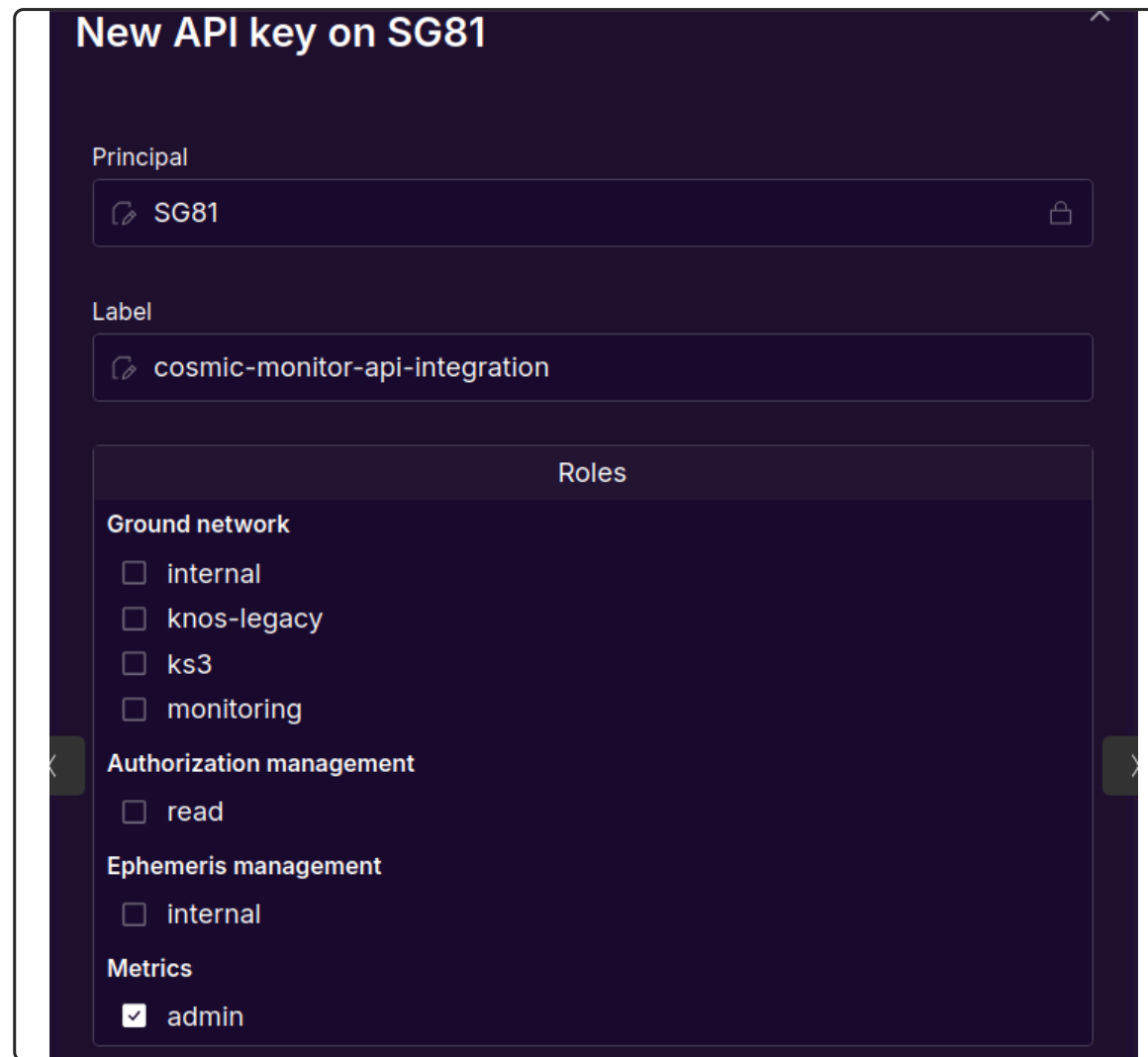
Required variables

VARIABLE	TYPE	DEFAULT
<code>cosmic_port</code>	string	<code>'8080'</code>
<code>cosmic_host</code>	string (ip address)	N/A . IP-Address of the cosmic host on the Site
<code>system_id</code>	uuid from KOGS	<code>'00000000-0000-0000-0000-000000000000'</code>

Optional variables

VARIABLE	TYPE	DEFAULT VALUES
<code>cosmic_api_port</code>	string	<code>'5000'</code>
<code>proxy</code>	string	<code>'http://proxy.ksat.site:3128'</code>
<code>morpheus_connect_timeout</code>	string	<code>'30'</code>
<code>morpheus_read_timeout</code>	string	<code>'30'</code>

KEY	DESCRIPTION
cosmic_monitor_api_key	made in kogoni. Must have Metrics - admin role
morpheus_password	Request morpheus user ANT1@ksat.no , they will give you a password to add to the vault.



How to test

Electron Control System

Electron Architecture

Scenario - Lite S/X

Scenario - Lite Ka

Spaceport

Electron and spaceport extensions

Electron - Scheduler Camriere and ...

Electron - Scheduler brokkr extensi...

Electron - ingest api extension

Electron - NAOS extension

Electron - Receptor gathering exte...

Spaceport - Monitor API with Cosm...

Spaceport - Lite Monitor and Contr...

Spaceport - SpaceNintendo GUI

MEOS Control Systems

Architecture

Falcon

Hypermatter

Girodyne

Observability

Grafana

OpenTelemetry

Deployment

GN Products Deployment Services

Deployment with AWS

Deployment to Openstack

Quick Guide - Inventory and Deplo...

How to make api key in KOGNI for Ele...

Vault

Michael.Johansen@ksat.no

Monitor api v2 is documented [here](#) (currently only preview).

Useful link for testing control api:

https://lite.gitlab.ksat.no/antenna_stack/control_api/quickguide.html#internal.

If you want to test using site proxy, use the following link:

https://lite.gitlab.ksat.no/antenna_stack/control_api/quickguide.html#external

If you want to test the monitor api, you can use the following script when inside Spaceport. There needs to be an active contact in Kogni to test. If you want to test "the customer way" through site proxy, change from

'localhost:80' to 'site_proxy_ip:8079' and ensure the contact is **not** booked as "internal contact".

```
import json
import asyncio
from logging import error
import websockets
from websockets.asyncio.client import connect
import os

api_key = os.environ.get("blablablanotneeded") #for backwards compatibility reasons. Cosmic mo
nitor api integration containers handles this
base_url = os.environ.get("BASE_URL", "ws://localhost:80/monitor/websocket/receive")
async def run():
    headers = {
        "Authorization": f"KSAT1-PLAIN {api_key}"
    }
    contact_map = {}
    # Stream will contain metrics for all available contacts and antennas
    async with connect(f"{base_url}", additional_headers=headers) as websocket:
        try:
            while True:
                data = await websocket.recv()
                data = json.loads(data)
                print(data)
        except websockets.ConnectionClosed:
            print("connection closed")
```